

DAY 19 ASSIGNMENT

COOKIES, SESSIONS & BROWSER SECURITY FUNDAMENTALS

Name:- Rishabh Raj

1. Why does a web application need state-management mechanisms if HTTP is stateless?

HTTP is stateless, meaning each request is treated independently and the server does not automatically remember previous requests. State-management mechanisms such as cookies and sessions allow a web application to remember information such as a logged-in user's session across multiple requests.

2. What is a cookie?

A cookie is a small piece of data stored by a browser for a website. It is sent back to the website with applicable requests and can be used for preferences, tracking, and maintaining application state such as a login session.

3. What is Set-Cookie?

Set-Cookie is an HTTP response header used by a server to instruct the browser to create or update a cookie. The header can also specify security and lifetime attributes such as Secure, HttpOnly, SameSite, Max-Age, and Path.

4. What is the Cookie request header?

The Cookie request header is sent by the browser to a server and contains cookies that match the request's domain, path, security requirements, and other rules. It allows the server to receive previously stored cookie values.

5. Explain the basic cookie request/response flow.

First, the browser sends a request to a website. The server can respond with a Set-Cookie header, after which the browser stores the cookie. On later matching requests, the browser sends the cookie back in the Cookie request header, allowing the server to use that state.

6. What is a web session?

A web session is server-side state associated with a particular user's interaction with an application. A session commonly stores information such as the authenticated user and other temporary state while the browser holds a session identifier.

7. What is a session ID?

A session ID is a unique, unpredictable identifier that links a browser's requests to a server-side session. It should be protected because possession of a valid session ID can allow an attacker to act as the associated user.

8. Explain the basic login-session flow.

The user submits login credentials over a protected connection. After successful authentication, the server creates or associates a session and sends a session identifier to the browser, often in a cookie. The browser sends that identifier on later requests, and the server uses it to identify the authenticated session.

9. Why should sessions expire?

Sessions should expire to reduce the period during which a stolen or forgotten session identifier can be abused. Expiration also limits the lifetime of inactive sessions and reduces the impact of session theft.

10. What happens conceptually during logout?

During logout, the application should invalidate the user's server-side session so the old session identifier can no longer authenticate requests. The browser may also be instructed to remove or expire the associated session cookie.

11. What is the Secure cookie attribute?

The Secure attribute tells the browser to send the cookie only over HTTPS connections. This helps prevent the cookie from being transmitted over unencrypted HTTP.

12. What is HttpOnly?

HttpOnly prevents JavaScript running in the page from directly reading the cookie through APIs such as `document.cookie`. It helps reduce the risk of client-side scripts stealing sensitive session cookies.

13. What is SameSite?

SameSite controls when a browser sends a cookie with cross-site requests. It can help reduce cross-site request forgery (CSRF) risks by restricting cross-site cookie transmission.

14. Explain Strict, Lax and None at a beginner level.

Strict is the most restrictive option and generally prevents the cookie from being sent with cross-site requests. Lax allows cookies in some top-level cross-site navigation scenarios and is a common balanced setting. None permits cross-site cookie use, but it requires the Secure attribute in modern browsers.

15. What do Expires and Max-Age control?

Expires specifies a date and time when a cookie should expire, while Max-Age specifies how many seconds the cookie should remain valid. These attributes control the cookie's lifetime.

16. What is Path used for?

Path limits the URL paths for which a cookie is sent. For example, a cookie with Path=/account is intended to be sent for requests under the /account path and its subpaths.

17. What is localStorage?

localStorage is browser storage that allows a website to store key-value data persistently on the user's device. The data generally remains available after the browser is closed until it is cleared by the site or user.

18. What is sessionStorage?

sessionStorage is browser storage for key-value data that is associated with a particular browser tab or window session. It normally remains available while that tab is open and is cleared when the tab or browsing context ends.

19. Why is sessionStorage different from a server-side login session?

sessionStorage is browser-side storage and does not automatically create or represent an authenticated server-side session. A server-side login session is maintained by the application and is used by the server to make authentication and authorization decisions.

20. Why should real session cookies never be shared in assignments or screenshots?

A real session cookie can act like a credential because it may allow access to an already authenticated account. Sharing it could expose the account to session hijacking, so assignments should use fictional values or harmless training cookies and must never reveal real authentication tokens.